

Application design for AI workloads on Azure

This article describes common design areas and factors to consider when you make decisions about technology and approach when you create an application that has AI functions. The guidance applies to both traditional machine learning and generative AI workloads. It covers frameworks like TensorFlow, Keras, and PyTorch, along with various serving and hosting options that include batch, stream, and online processing.

There are many choices to consider when you design intelligent capabilities into your workload. Your unique functional and nonfunctional requirements help you determine whether you need basic inferencing capabilities or more dynamic problem-solving capabilities. These requirements guide your high-level design decisions. Consider these choices as you move from high-level design areas to lower-level design areas.

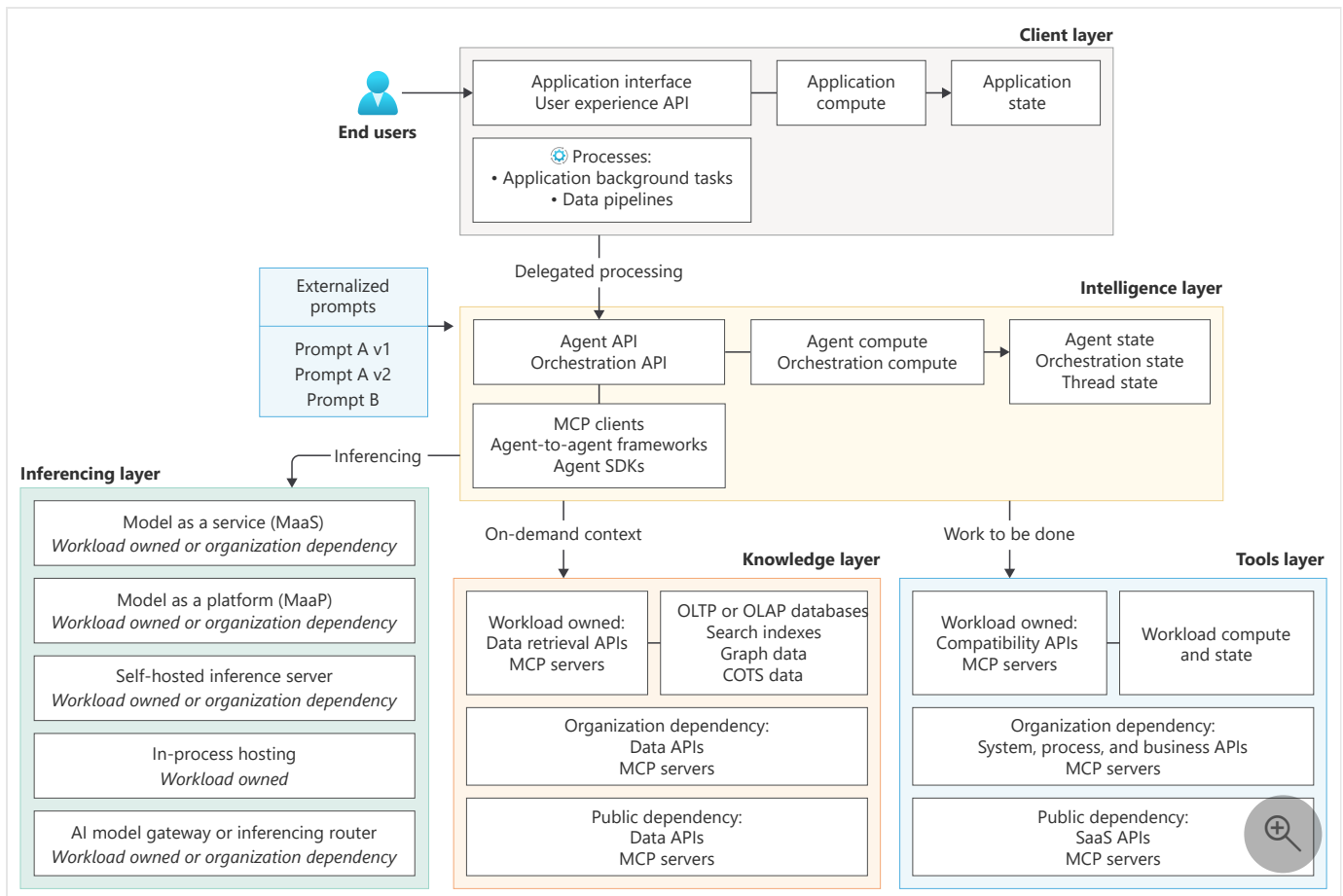
As discussed in the [Get started](#) article, whether to build your own model or use a prebuilt model is one of the first important decisions to make. When you use a prebuilt model, consider the following points:

- **Catalog sources:** Explore repositories like the Hugging Face Model Hub, TensorFlow Hub, and the [Microsoft Foundry portal](#)  model catalog to find pretrained models. These platforms provide an extensive catalog of models for various tasks.
- **Licensing:** Make sure that the model's licensing terms fit your security, compliance, and application goals, especially if you plan to distribute the application or integrate it with other services.
- **Key components:** Look at the model's architecture, training data, and performance to determine whether it's fine-tuned for your task or domain.

For guidance about how to choose a hosting platform, see [Considerations for the model hosting and inferencing platform](#).

Application layer architecture

When you design intelligent capabilities, establish clear boundaries in your design across the following five key layers.



- **Client layer:** The user interface and client applications where users or processes experience your workload's intelligent capabilities. Keep this layer thin and delegate most capabilities to other layers.
- **Intelligence layer:** Routing, orchestration, and agent capabilities that coordinate AI operations. This layer includes model routing, conversation management, and intelligent decision-making.
- **Inferencing layer:** Running trained models to generate predictions or decisions based on data. This layer handles model loading and runtime invocation, input preprocessing and output postprocessing, along with serving predictions via APIs or embedded systems.
- **Knowledge layer:** Grounding data, knowledge graphs, and retrieval services that provide relevant context and information to the intelligence layer. This layer enforces data access policies and authorization.
- **Tools layer:** Business APIs, external services, and action capabilities that the intelligence layer can invoke. This layer should use standardized interfaces and enforce its own security policies.

Each layer enforces its own policies, identities, and caching strategies to achieve their own localized reliability, security, and performance requirements. The layers also enable focused

development, testing, and troubleshooting.

Recommendations

The following table summarizes the recommendations provided in this article.

 Expand table

Recommendation	Description
Prioritize security and Responsible AI controls.	Implement traditional application security plus AI-specific safety measures as a primary design driver. Enforce provider safety systems, input/output filtering, identity-bound rate limiting and quotas, and token/prompt caps. Security and safety controls must be verified and can't be assumed from managed services.
Keep intelligence away from the client.	Design back-end services to handle cross-cutting concerns like rate limiting, failover operations, and AI processing logic. Abstract behavior and intelligence away from the client to future-proof your design and improve maintainability.
Block direct access to data stores.	Code in AI systems shouldn't directly access your data stores. Route all data requests through an API or similar data access abstraction that enforces authorization and propagates user or tenant context into retrieval and filtering. Pass forward user identity so that data level security can be applied.
Abstract your models and tools.	Use abstraction layers to decouple your application from specific models, tools, and technologies. Implement standardized interfaces and protocols to provide flexibility as technologies evolve, which makes your design more maintainable and future-proof.
Isolate behaviors and actions.	Design clear boundaries across client, intelligence (routing/orchestration/agents), knowledge (grounding data), and tools (business APIs) layers. Each layer should enforce its own policies, identities, and caching strategies to reduce blast radius and focus development efforts.
Prioritize prebuilt solutions.	Use software as a service (SaaS) or platform as a service (PaaS) to handle workload functions when they meet security, safety, compliance, and quota needs. Implement compensating controls through gateways that enforce authentication, quotas, safety, and logging. Use prebuilt and pretrained models where possible to minimize the operational and development burden for your workload and operations teams.

Distinguish between inferencing and intelligent applications

When you design your application architecture, first determine whether you're building an inferencing-focused application or an intelligence-focused application because this distinction

drives design decisions.

Inferencing applications

Inferencing applications perform single-step operations like classification, translation, or summarization. These applications have simpler architectures.

Typical architecture: The client communicates with an AI gateway that provides authentication, quotas, safety, and routing. That gateway calls into the model serving layer such as Foundry, Azure Kubernetes Service (AKS), or managed online endpoints. Where practical, the results might be cached for future inferencing calls before being returned to the client.

Inferencing applications typically feature direct model calls, require minimal orchestration, focus on performance and throughput, and don't require complex state management.

Intelligent applications

Intelligent applications perform planning, coordination, and multi-step reasoning and are typically handled through agents and agent orchestration. They require more concepts to be addressed in their architectures.

Typical architecture: The client invokes an agent or agent orchestrator. Based on the design or autonomous capabilities, this layer invokes a tools layer such as Model Context Protocol (MCP) servers or custom APIs. The agent might need to call into grounding knowledge services such as a search index, database, or graph. Models might be invoked at multiple points during this process. Caching can also occur at multiple levels to optimize the process.

Intelligent applications typically feature agentic patterns and model routing with complex workflow coordination, integrate with multiple data sources and tools, and require conversation and context management.

Fundamental AI application design guidelines

When you design AI applications, apply the following foundational guidelines to create resilient and maintainable systems.

Start with business outcomes

Before you select technologies, clearly define the business problem you're solving. This definition guides technology choices and architecture design. Consider the following factors:

- **Success metrics:** Define measurable outcomes that demonstrate value, such as accuracy improvements, cost reductions, or user satisfaction scores.
- **User experience requirements:** Understand how users and processes will interact with AI capabilities and what response times that they expect.
- **Regulatory constraints:** Identify compliance requirements that might affect your design choices, such as data residency or explainability requirements.

Design for security from day one

AI workloads demand a security-first approach from the outset. Implement layered security across all components by using identity propagation and auditable controls. For more information, see [Security design strategies](#).

Design for observability from day one

AI applications require monitoring beyond traditional application metrics. Build observability into your design from the start:

- **Model performance tracking:** Monitor accuracy drift, inference latency, and prediction confidence scores.
- **Data quality monitoring:** Track input data distribution changes that might affect model performance.
- **User interaction analysis:** Understand how users interact with AI features to identify improvement opportunities.

Plan for abstraction and future flexibility

Design your system with abstraction layers that allow you to adapt as AI technologies evolve:

- **Model abstraction:** Abstract models behind consistent interfaces to enable swapping or upgrading without application changes.
- **Standardized protocols:** Prefer open, documented interfaces and formats such as OpenAPI for tools, ONNX for model portability, and OpenTelemetry for telemetry.
- **Capability-based design:** Design around capabilities rather than specific technologies to maintain flexibility.

Externalize prompts and configuration

Treat prompts as configuration that should be externalized according to [The Twelve-Factor App](#) design principles:

- **Version control:** Keep prompts in version control with clear deployment tracking to correlate telemetry and safety results with specific prompt versions.
- **Role separation:** Enable business analysts and domain experts to tune prompts without code changes.
- **Safe deployment:** Implement controlled rollout processes for prompt changes and treat them as significant configuration updates.

Plan for model deprecation

Foundation models eventually reach end-of-life and are retired by your model hosting platform. The models are replaced by models that perform better, are more cost effective, have updated training knowledge, and support new capabilities. Design abstractions that minimize the impact of your workload's future model transitions.

- **Provider abstraction:** Use abstraction layers that allow switching between model providers without application changes.
- **Version management:** Implement versioning strategies that support gradual migration between model versions.
- **Fallback strategies:** Design fallback mechanisms for when preferred models become unavailable.

For architecture design techniques that address model life cycle concerns, see [Design to support foundation model life cycles](#).

Containerize components

To ensure that your independently deployable components are self-contained and to improve deployment consistency, consider containerization as part of your design strategy. The following components should be containerized:

- **Microservices:** Containerize individual microservices that handle specific functions of the application, like data processing, model inference, and user authentication. This approach

enables independent deployment and scaling and facilitates more efficient updates and maintenance.

- **AI models:** Containerize AI models to ensure that all dependencies, libraries, and configurations are bundled together. This approach isolates the model environment from the host system to prevent version conflicts and help ensure consistent behavior across different deployment environments.
- **Data processing pipelines:** Containerize data processing tasks that precede or follow model inference, like data cleaning, transformation, and feature extraction. This approach enhances reproducibility and simplifies the management of dependencies.
- **Infrastructure services:** Containerize services that provide infrastructure support, like databases and caching layers. This approach helps maintain version consistency and facilitates easier scaling and management of these components.

Implement multi-layer caching strategies

A multi-layer caching approach can help improve performance and reduce costs in your AI applications. Consider implementing caching at multiple levels of your application stack:

- **Result and answer caching:** Use this approach to reuse responses for identical or semantically similar queries, if appropriate for the situation. In workloads where responses are cacheable, this approach can significantly reduce invocations to your model, which improves performance and reduces model invocation costs.
- **Retrieval and grounding snippet caching:** Cache frequently retrieved knowledge fragments and grounding data to avoid repeated database and search queries or data API operations.
- **Model output caching:** Cache intermediate model outputs that can be reused across requests.

When you design your caching approach in each layer, you need to understand what data is frequently accessed and how the application's users' roles and permissions might influence what data they can access. For caching to be effective, you must expect a healthy cache hit rate. Focus on caching for high-traffic paths like product catalog searches, while monitoring cache evictions.

- **Cache key components:** Cached values need to be tied to specific runtime factors within your workload. Include values such as tenant or user identity, policy context, model version, and prompt version in cache keys to ensure that a cached value is only being returned when appropriate for the request.

- **Time-to-live (TTL) policies:** Set appropriate expiration times based on data freshness requirements and content sensitivity.
- **Invalidation hooks:** Implement cache invalidation triggers for data updates, model changes, and prompt modifications.
- **User privacy protection:** Never cache user-private content unless properly scoped by key and policy. In general, caching works best for data that applies across multiple users. Don't cache for a single user in most situations. Avoid caching user-specific responses as a valid response for all users. For example, a cached response for "How many hours of paid time off do I have left?" is only ever appropriate for the user that requests the data, even if other users present the same query.



Risk: Caching improves performance and reduces costs but introduces security and data freshness risks. These risks include **data leakage**, **stale data**, and **privacy violations**.

Evaluate the use of orchestration and agents in generative AI solutions

Choose the appropriate coordination approach based on your application's requirements for determinism and exploratory capabilities.

When to use orchestration

An orchestrator manages a workflow by coordinating the communication among the different components of the AI solution that would otherwise be difficult to manage in complex workloads. We recommend that you build an orchestrator into your design if your workload needs any of the following characteristics:

- **Predictable workflows:** Multi-step processes with well-defined sequences and decision points.
- **Compliance requirements:** Scenarios where you need to ensure that specific steps are performed for regulatory compliance.
- **Performance-critical paths:** Workflows where latency and resource usage must be tightly controlled.

- **Simple coordination:** Straightforward task delegation that doesn't require dynamic reasoning.

When to use agent collaboration

An **agent** is a way to wrap, extract, and define intelligent behavior in your application. Agents provide context-bound functionality and can work with orchestration frameworks like Semantic Kernel, Autogen, or Microsoft Agent Framework.

Use agent-routed collaboration approaches for exploratory or composed tasks:

- **Multi-step reasoning:** Tasks that require planning and decision-making across multiple steps
- **Tool coordination:** Complex workflows that involve coordinating multiple specialized tools and services
- **Adaptive behavior:** Scenarios where the system needs to adjust its approach based on intermediate results or changing conditions
- **Context management:** Applications that need to maintain conversation state and user context across interactions

Agent design considerations

When you design agent-based systems, consider isolation and boundaries, communication patterns, and tool abstraction strategies. Design clear boundaries between different agent capabilities to reduce blast radius and improve testability. Use established patterns like topic-queue systems for agent communication. Abstract tool capabilities by using standardized interfaces to enable agent flexibility.

Important

Don't automatically add agents between the task to be completed and model calls. Evaluate whether the intelligence being delivered requires the complexity of agent patterns, or if direct model calls are sufficient for your use case. Agent layers add latency, expand the surface area, and complicate testing.

Agent state persistence

For multi-turn conversations and long-running tasks, design agents that include durable state persistence so that they can retain context across requests, sessions, or deployment cycles. Use secure, shared storage such as Azure Cosmos DB, Azure Managed Redis, or Azure Table Storage to store relevant metadata, conversation history, and task progress. Scope persisted state to the minimal necessary information to reduce privacy risks and token overhead, and implement TTL policies to expire stale data. Ensure that agents can rehydrate state on resumption to continue workflows without repeating completed steps.

Hybrid approaches

Consider hybrid designs where orchestrators delegate specific subtasks to agents.

- **Structured workflows with flexible steps:** Use orchestration for the overall workflow while allowing agents to handle complex individual steps.
- **Escalation patterns:** Start with deterministic orchestration, and escalate to agent-based reasoning when predetermined logic is insufficient.
- **Domain-specific reasoning:** Use orchestration for cross-domain coordination while employing specialized agents for domain-specific tasks.



Trade-off: Orchestration provides predictability and control but limits adaptability. Agent collaboration enables dynamic problem-solving but introduces variability and complexity.

Implement AI gateways for policy enforcement

Use AI gateways in your design to provide [gateway offloading](#), [gateway routing](#), and [gateway aggregation](#) capabilities within your workload. Gateways are typically implemented by using platforms like Azure API Management or through custom-coded solutions, such as an Envoy or NGINX implementation. Gateways can be used to proxy requests to model providers, agent endpoints, tools, and knowledge stores.

Core AI gateway capabilities

An AI gateway can address cross-cutting concerns and serve as a layer of abstraction and indirection from the target system. When you design an AI gateway strategy, consider the following capabilities:

- **Protocol normalization:** For shared AI capabilities across providers, normalize interfaces to a common protocol via the gateway to improve maintainability. For provider-specific features, use the gateway as a reverse proxy without normalization.
- **Authentication and authorization:** Enforce identity-based access control, and propagate user context to downstream services.
- **Rate limiting and quotas:** Implement per-user, per-application, or per-tenant rate limits and token quotas to prevent abuse and manage costs.
- **Request and response filtering:** Apply input validation, prompt filtering, and output safety checks at the gateway level.
- **Token and prompt caps:** Enforce maximum token limits and prompt size restrictions to prevent resource exhaustion by preemptively rejecting requests that represent more demand on the system than desired or possible.
- **Model routing policies:** Route requests to appropriate models based on user permissions, request characteristics, or cost optimization goals. Gateways are often used to implement spillover between multiple models when capacity or pricing is consumed on one model but is available on another model deployment.
- **Header injection and transformation:** Add required headers, user context, and security tokens for downstream services.
- **Chargeback management:** Allocate charges across departments that share AI workload components, like a single Azure OpenAI instance.

**Tip**

For an in-depth look at how an AI gateway can be used for cross-cutting concerns and how a gateway introduces extra complexity into your architecture, see [Access Azure OpenAI and other language models through a gateway](#).

Multi-provider scenarios

AI gateways are especially valuable when you use multiple model providers or shared model hosting platforms. These gateways are often centralized resources within your organization that your workload is encouraged or required to depend on. A sufficiently complex workload might also benefit from building this layer of abstraction within its own design.

- **Unified interface:** Provide a single API surface that abstracts multiple back-end providers.
- **Failover and redundancy:** Route requests to alternate providers when primary services are unavailable.
- **Cost optimization:** Route requests to the most cost-effective provider based on request characteristics.

Important

Don't rely solely on SDK retries and timeouts. Enforce limits, authentication, quotas, and timeouts at the gateway and policy layer for comprehensive control.

Use AI application design patterns

The industry defines several common design patterns for AI applications. You can use them to simplify your design and implementation. These design patterns include the following types:

- **Model ensembling:** This design pattern combines predictions from multiple models to improve accuracy and robustness by mitigating the limitations of individual models.
- **Microservices architecture:** This design pattern separates components into independently deployable services to enhance scalability and maintainability. It enables teams to work on different parts of the application simultaneously.
- **Event-driven architecture:** This design pattern uses events to trigger actions, which enables decoupled components and real-time processing to make the system more responsive and adaptable to changing data.

Grounding and knowledge integration patterns

Many AI applications access external knowledge sources to provide accurate, up-to-date information. Treat retrieval as an agent tool or knowledge call behind a service that enforces authorization.

- **Knowledge routing:** Direct queries to appropriate knowledge sources based on query type, domain, or user context.
- **Context layering:** Build systems that can combine multiple knowledge sources and context types to provide comprehensive responses.

- **Dynamic grounding:** Implement systems that can adapt their knowledge sources and grounding strategies based on changing requirements or data availability.
- **Authorization-aware retrieval:** Ensure that grounding services enforce user permissions and tenant context when retrieving information.

For a deeper understanding of designing a purpose-built vector index that supports your workload-specific grounding data for semantic retrieval, see [Design and develop a retrieval-augmented generation \(RAG\) solution](#).

Multi-model routing patterns

Using a model router can improve your workload's availability by routing requests to a healthy model when another model is in an unhealthy state, or it can help improve the quality of responses by selecting the best model for a specific task in real time. Model routers add flexibility but introduce extra variability. Use them selectively:

When to use model routers: Use a model router when your workload can tolerate added variability and latency, the user experience expects breadth across model types, or you need to balance costs and capabilities across different models.

When to avoid model routers: Don't use a model router in scenarios where the workload needs precise answers optimized for specific tasks, when you use fine-tuned models with narrow service-level objectives (SLOs), or when deterministic behavior, including consistent performance, is critical for your use case.

When you add a model router to your application, prefer provider-native routing when available, and monitor performance impact and user satisfaction.



Trade-off: Model routers provide flexibility and cost optimization but add non-deterministic behavior and complexity to testing and troubleshooting.

If your workload is a multitenant application, see [Guide to design a secure multitenant RAG inferencing solution](#) for recommendations about accessing per-tenant grounding data.

When to use design patterns

Consider using these design patterns for the following scenarios:

- **Complex workflows:** When you have complex workflows or interactions between multiple AI models, patterns like RAG or microservices can help manage complexity and ensure clear communication between components.
- **Scalability requirements:** If the demand on your application fluctuates, a pattern like microservices enables individual components to scale independently to accommodate varying loads without affecting the overall system performance.
- **Data-driven applications:** If your application requires extensive data handling, an event-driven architecture can provide real-time responsiveness and efficient data processing.

ⓘ Note

Smaller applications or proof of concepts (POCs) typically don't benefit from these design patterns. These applications should be designed for simplicity. Likewise, if you have resource constraints, such as budget, time, or headcount, using a simple design that can be refactored later is a better approach than adopting a complex design pattern.

Choose the right frameworks, libraries, and protocols

The choice of frameworks and libraries is closely intertwined with application design. They affect performance, scalability, and maintainability. However, design requirements can limit your framework choices. For example, use of the Semantic Kernel SDK often encourages a microservices-based design where each agent or function is encapsulated within its own service. Consider the following factors when you choose frameworks and libraries:

- **Application requirements:** The requirements of the application, like real-time processing or batch processing, might limit the choice of framework. For example, if the application requires low latency, you might need to use a framework that has asynchronous capabilities.
- **Integration needs:** The design might require specific integrations with other systems or services. If a framework doesn't support the necessary protocols or data formats, you might need to reconsider the design or choose a different framework.
- **Team expertise:** The skill set of the development team can limit framework choices. A design that relies on a less familiar framework might lead to increased development time and complexity, so consider using a more familiar tool.

- **Community and support:** Consider the maturity and support ecosystem of the framework. Active communities and comprehensive documentation reduce implementation risks.
- **Performance characteristics:** Evaluate framework performance for your specific use case, including memory usage, startup time, and inference speed.

Adopt standardized tool protocols to improve governance and enable flexibility:

- **Use MCP-style servers:** Use MCP or another suitable standard to wrap business capabilities as discoverable tool servers.
- **Use specifications that have broad SDK support.** For example, define tools by using OpenAPI specifications for consistent interface documentation and validation.
- **Advertise capabilities:** Design tools to advertise their capabilities, which allows orchestrators to discover and route requests appropriately. For example, wrap enterprise resource planning (ERP) read and write operations as a tool server that advertises capabilities. This approach allows you to tune the ERP interactions without changing the agent logic.

Protocol preferences

When you select protocols and standards, prefer open, documented interfaces over proprietary formats. There are a lot of emerging protocols in the industry. As you consider your agent-to-agent and control use cases, consider technical debt that can come from protocols that are under rapid evolution or deprecation. Layers of abstraction can help keep your design interoperable across different AI frameworks and providers.

Design a security strategy for your workload's AI components

AI components require security considerations beyond traditional application security. Implement security measures across all application layers while using standard identity providers like Microsoft Entra ID for foundational authentication and authorization.

Core security principles

Implement the following foundational security practices:

- **Standard authentication and authorization:** Use established identity providers and role-based access control (RBAC) systems.
- **Data lineage security with user context propagation:** Ensure that access controls are maintained throughout the data pipeline. Pass forward user identity by using on-behalf-of flows so that users can only access results based on data that they're authorized to see.
- **Audit trails:** Implement detailed logging of AI interactions for compliance and security monitoring.

AI-specific security measures

Address security challenges that are unique to AI applications:

- **Prompt filtering and injection prevention:** Implement safeguards against prompt injection attacks that can manipulate AI behavior or extract sensitive information.
- **Safe tool usage controls:** When you use agents with tool access, implement controls to prevent harmful actions and validate tool usage before invocation.
- **Agent behavior monitoring:** Monitor agent actions and decisions to detect unusual or potentially harmful behavior patterns.
- **Model access control:** Implement fine-grained permissions for different models and capabilities within your AI system.
- **Response filtering and safety checks:** Filter AI outputs to prevent the generation of harmful, inappropriate, or sensitive content.

Security-trimmed grounding

Go beyond preventing direct database access, and implement security-aware knowledge retrieval:

- **Identity propagation:** Pass Microsoft Entra group claims or equivalent identity information into knowledge tool calls so that grounding services never retrieve unauthorized data or media.
- **Group and access control list (ACL)-based trimming:** Implement filtering based on security groups and ACLs at the knowledge layer.

- **Auditable denials:** Log and audit logging request denials caused by insufficient permissions.
- **Authorization enforcement:** Require that code sits between agents and knowledge sources to enforce authorization policies.

For example, you might pass Microsoft Entra group claims into the knowledge tool call to ensure that grounding never retrieves unauthorized documents. This configuration guarantees cached results and ensures that AI responses respect user permissions.

Consider nonfunctional requirements

Your workload might have nonfunctional requirements that pose challenges because of factors that are inherent to AI technologies.

- **Latency of model inferencing or timeouts:** AI applications often require real-time or near-real-time responses. Designing for low latency is crucial. It involves optimizing model architecture, data processing pipelines, and hardware resources. Implementing caching strategies and ensuring efficient model loading are also essential to avoid timeouts and provide timely responses.
- **Token or request throughput limitations:** Many AI services impose limits on the number of tokens or the throughput of requests, especially with cloud-based models. Designing for these limitations requires careful management of input sizes, batching of requests when necessary, and potentially implementing rate-limiting or queuing mechanisms to manage user expectations and prevent service disruptions.
- **Cost and chargeback scenarios:** Designing for cost transparency involves implementing usage tracking and reporting features that facilitate chargeback models. These features enable your organization to allocate costs accurately across departments. Chargeback management is typically handled by an API gateway, like [API Management](#).
- **Model accuracy and drift:** AI models can degrade over time as data patterns change. Design monitoring systems to detect accuracy drift, and implement automated retraining pipelines when necessary.
- **Compliance and explainability:** Some industries require explainable AI decisions. Design your system to capture and provide reasoning for AI-generated outputs when required.
- **Data privacy and residency:** Ensure that your design meets data protection requirements, including the ability to delete user data and comply with geographic data residency rules.

Next step

Design area: Application platform

Last updated on 10/08/2025